

EXPERIMENT - 2

A) Aim:

To study and demonstrate the different stages of compilation of a C program using the -save-temps option and to execute a C program for calculating the area of a circle.

B) Requirements:

- 1) Linux environment (Kali Linux / CoCalc terminal).
- 2) GCC (GNU C Compiler).
- 3) A C source file named area.c.
- 4) Terminal for compiling and executing the program.

C) Theory:

A C program passes through several stages

before it is executed. The main stages are preprocessing, compilation, assembly and linking.

The GCC compiler can preserve the intermediate files by using the `-save-temps` option. For a source file `area.c`, the command below keeps the intermediate files generated during compilation:

```
gcc -save-temps area.c -o area
```

The important files produced are:

1) `area.c` – C source code written by the programmer.

2) `area.i` – Preprocessed C source file. Header files are expanded and preprocessing directives are handled.

3) `area.s` – Assembly language code

generated from the C program.

4) area.o – Object file containing machine-level code that is not yet linked.

5) area - Final executable program produced after linking.

D) C Program to Calculate Area of a Circle

Formula used:

$$\text{Area} = \pi \times r \times r$$

The following program takes the radius as input and calculates the area of the circle.

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    float radius, area;
```

```
printf("Enter radius of circle: ");  
  
scanf("%f", &radius);  
  
area = 3.14159 * radius * radius;  
  
printf("Area of circle = %.2f\n", area);  
  
return 0;  
  
}
```

E) Procedure and Compilation Stages:

Step 1: Create the C source file

Create a file named area.c and enter the C program given above.

nano area.c

Save the file after entering the program. You can also use vi or another text editor.

```
#include<stdio.h>
#define PI 3.14
int main(void)
{
    int radius;
    float area;
    printf("Enter the radius: ");
    scanf("%d", &radius);
    area = PI*radius*radius;
    printf("Area = %f\n", area);
    return 0;
}
```

Step 2: Compile using -save-temps:

Use the following command. The -save-temps option tells GCC to keep the intermediate files.

```
gcc -save-temps area.c -o area
```

After successful compilation, GCC keeps the intermediate files such as area.i, area.s and area.o, along with the final executable area.

ls

Show the compilation command and the generated files using ls.

```
(kali㉿kali)-[~/Desktop]
$ nano area.c

(kali㉿kali)-[~/Desktop]
$ cc -save-temps area.c

(kali㉿kali)-[~/Desktop]
$ ls
a-area.i  a-area.o  a-area.s  a.out  area.c

(kali㉿kali)-[~/Desktop]
$
```

Step 3: Preprocessing stage – area.i:

In preprocessing, header files are expanded and preprocessing directives such as `#include` are processed. The result is stored in `area.i`.

```
gcc -E area.c -o area.i
```

To view the beginning of the preprocessed file:

```
head area.i
```

Show the `area.i` file or its output using `head`.

```
(kali㉿kali)-[~/Desktop]
$ head a-area.i
# 0 "area.c"
# 0 "<built-in>"
# 0 "<command-line>"
# 1 "/usr/include/stdc-predef.h" 1 3 4
# 0 "<command-line>" 2
# 1 "area.c"
# 1 "/usr/include/stdio.h" 1 3 4
# 28 "/usr/include/stdio.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 1 3 4
# 33 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 3 4

(kali㉿kali)-[~/Desktop]
$ less a-area.i

(kali㉿kali)-[~/Desktop]
$ head a-area.i
# 0 "area.c"
# 0 "<built-in>"
# 0 "<command-line>"
# 1 "/usr/include/stdc-predef.h" 1 3 4
# 0 "<command-line>" 2
# 1 "area.c"
# 1 "/usr/include/stdio.h" 1 3 4
# 28 "/usr/include/stdio.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 1 3 4
# 33 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 3 4

(kali㉿kali)-[~/Desktop]
$
```

Step 4: Compilation stage – area.s

The compiler converts the preprocessed C program into assembly language instructions. The assembly code is stored in area.s.

```
gcc -S area.c -o area.s
```

To view the assembly file:

head area.s

Show area.s containing assembly instructions.

```
(kali㉿kali)-[~/Desktop]
$ head a-area.s
        .file      "area.c"
        .text
        .section   .rodata
.LC0:
        .string   "Enter the radius: "
.LC1:
        .string   "%d"
.LC3:
        .string   "Area = %f\n"
        .text
(kali㉿kali)-[~/Desktop]
$
```

Step 5: Assembly stage – area.o

The assembler converts the assembly language code into machine-level object code. The output is stored in the object file area.o.

```
gcc -c area.s -o area.o
```

The file command can be used to identify the object file:

file area.o

Show the creation/identification of area.o using the terminal.

```
(kali㉿kali)-[~/Desktop]
$ ls -l a-area.o
-rw-rw-r-- 1 kali kali 1688 Aug 18 14:35 a-area.o

(kali㉿kali)-[~/Desktop]
$ file a-area.o
a-area.o: ELF 64-bit LSB relocatable, x86-64, version 1 (SYSV), not stripped

(kali㉿kali)-[~/Desktop]
$
```

Step 6: Linking stage – final executable:

The linker combines the object file with the required libraries and produces the final executable program.

gcc area.o -o area

Check the generated executable:

ls -l area

Show the linking command and the final executable file named area.

```
(kali㉿kali)-[~/Desktop]
$ cc a-area.o -o area

(kali㉿kali)-[~/Desktop]
$ ls -l area
-rwxrwxr-x 1 kali kali 16008 Aug 18 14:50 area

(kali㉿kali)-[~/Desktop]
$ file area
area: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=748ef5cd59187f72be7580861f4f08ae593ee4a6, for GNU/Linux 3.2.0, not stripped

(kali㉿kali)-[~/Desktop]
$
```

F) Execution of the C Program:

Run the final executable using the following command:

`./area`

Example input and output:

Enter radius of circle: 5

Area of circle = 78.54

For radius = 5, the calculated area is approximately 78.54 square units.

Show the execution of `./area`, the entered radius and the calculated area.

```
(kali㉿kali)-[~/Desktop]
$ ./area
Enter the radius: 5
Area = 78.500000

(kali㉿kali)-[~/Desktop]
$
```

G) Result:

The stages of compilation of a C program were studied successfully using GCC and the -save-temps option. The

intermediate files area.i, area.s and area.o were generated, the final executable was created, and the program was

executed successfully to calculate the area of a circle.